allgather/recursive_doubling

An AgentMPI collective scaling analysis

Abstract

This family has nine members where its siblings have twenty-one, and all nine are powers of two. That absence is the result. Recursive doubling for allgather exchanges everything a rank holds with the partner at $r \oplus 2^k$ and doubles its holdings each round, which requires the partner to exist for every rank in every round — that is, requires p to be a power of two. When it is not, the implementation skips the missing exchanges, and skipping them does not merely cost extra rounds, it produces a wrong answer: simulating the block sets at $p = 10$, six of ten ranks finish without ranks 8 and 9's contributions. `algorithms.allgather` therefore rewrites the caller's request to `bruck` whenever $(p \,\&\, (p-1)) \neq 0$, and because that rewrite happens before the trace record is created, a sweep run at $p = 10$ would have been labelled `recursive_doubling` and contained `bruck`'s events; `microbench.py` skips those twelve cases explicitly rather than archive mislabelled data. Where the algorithm does run it is exactly correct — $p\lceil\log_2 p\rceil$ messages in $\lceil\log_2 p\rceil$ rounds, matching the closed form and the log at all nine sizes with no accounting gap — and exactly as expensive in messages and rounds as Bruck at the same p, because at powers of two the two algorithms are the same algorithm. They differ only in encoding: recursive doubling ships a dictionary keyed by rank index and Bruck a bare list, which costs it a factor of two on the wire (6,624 tokens against 3,360 at $p = 32$) and a largest single message of 106 tokens against 53. The takeaway is that this algorithm is dominated in the strict sense — never cheaper, applicable at 9 sizes instead of 21 — and that its family exists mainly to establish the boundary of its applicability, which is a real property of an algorithm and not a gap in the data.

1 What this family is

The `allgather` collective under the `recursive_doubling` algorithm, measured at 9 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 9 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.014–0.247 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

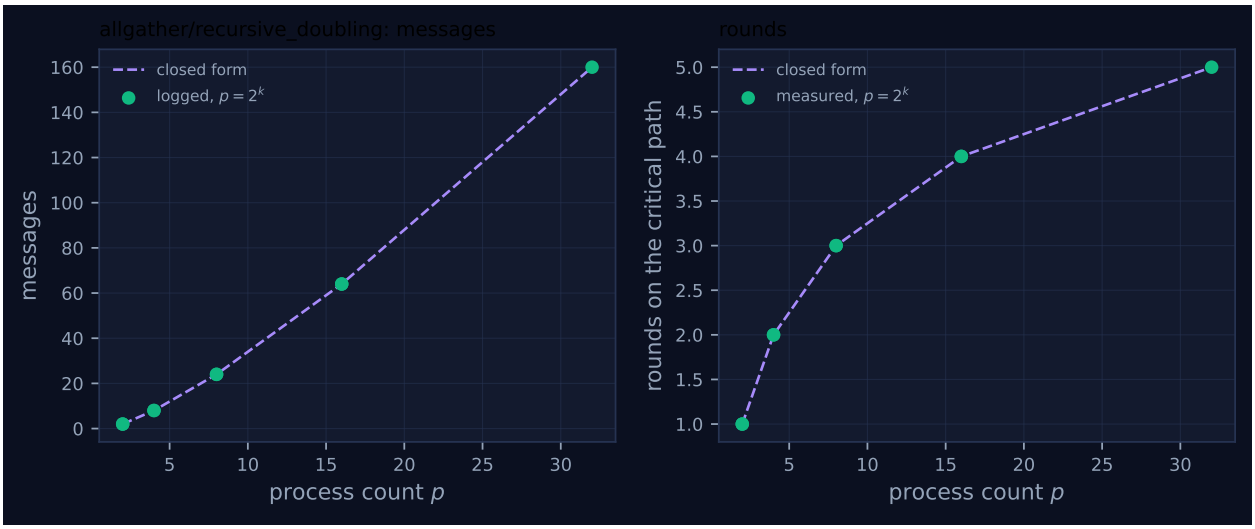

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.016	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.014	✓	✓
4	`mb-free`	8	8	8	2	2	0.024	✓	✓
4	`mb-smoke`	8	8	8	2	2	0.020	✓	✓
8	`mb-free`	24	24	24	3	3	0.057	✓	✓
8	`mb-smoke`	24	24	24	3	3	0.056	✓	✓
16	`mb-free`	64	64	64	4	4	0.175	✓	✓
16	`mb-smoke`	64	64	64	4	4	0.203	✓	✓
32	`mb-free`	160	160	160	5	5	0.247	✓	✓

4 How the algorithm scales

4.1 Where the cost comes from

The algorithm is the natural hypercube allgather. Rank r maintains a map from rank index to block, starting with its own. In round k it exchanges the *whole* map with $r \oplus 2^k$ and merges, so its holdings double: 1 block, then 2, then 4, and after $\log_2 p$ rounds all p. Because the partner relation is an involution on a hypercube, every rank sends in every round, giving $p \log_2 p$ messages in $\log_2 p$ rounds; and because rank r sends 2^k blocks in round k, each rank ships $\sum_k 2^k = p - 1$ blocks and the total volume is $p(p-1)n$ — the information-theoretic floor, since each rank must receive $(p-1)n$. The `cost.FORMULAS` entry $(\lceil \log_2 p \rceil, p\lceil \log_2 p \rceil, p(p-1)n, 0)$ is that derivation, and it is byte-for-byte the same expression as the Bruck entry.

The log confirms it. `mb-free__coll-allgather-recursive_doubling-32` holds 160 `msg.send` events in five tag phases of 32 each, carrying 7, 14, 27, 53 and 106 tokens per message — the

doubling sequence — for 6,624 tokens and $1 + 2 + 4 + 8 + 16 = 31 = p - 1$ blocks per rank. All nine sizes agree with both components of the closed form, and the self-reported count matches the logged traffic at all nine.

4.2 What the figure shows, and what it cannot

Figure 1 is the only figure in the twenty-two families with a single marker class. There are no squares, because there are no non-power-of-two sizes, and no red crosses, because the accounting is clean. The legend has two entries where its siblings have three or four.

This has an effect on the plot that is worth naming, because it inverts the usual reading. The dashed closed-form line is evaluated only at the measured process counts and joined by straight segments, and here the measured counts are $2, 4, 8, 16, 32$. So the rounds panel, whose true shape is a staircase with flat treads of length 2^{k-1}, renders as a smooth concave curve through five points — and every one of those points sits at the *left edge* of a tread. The staircase is invisible, and it is invisible precisely because the treads are the region where this algorithm cannot run. In the Bruck family the same function is plotted over the same range and appears, correctly, as a staircase. The same is true of the messages panel: $p\lceil \log_2 p \rceil$ jumps by 33% between $p = 16$ and $p = 17$, and with no measurement between them the line renders the jump as a gentle slope.

So the two visual claims one would normally make about a family figure — "the round count steps where $\lceil \log_2 p \rceil$ steps" and "the non-power-of-two sizes behave like the powers of two" — are respectively unverifiable and vacuous here. What the figure does establish is that the five points are exactly on the exact closed form, which is worth having and is all that is available.

5 Powers of two and everything else

5.1 Why there is no "everything else"

The restriction is two lines of `algorithms.allgather`. The exchange is guarded:

```
partner = r ^ (1 << k)
if partner >= p:  continue
```

so a rank whose partner does not exist skips that round. At a power of two no rank ever skips. At any other p some do, and the consequence is not a cost penalty but an incorrect result, because a skipped round is a subtree of the hypercube that never merges.

Simulating the block sets directly — iterating have$[r] \mid=$ have$[r \oplus 2^k]$ over $\lceil \log_2 p \rceil$ rounds with the same guard — gives the following count of ranks that finish without the full set:

p	incomplete ranks	example
5	3 of 5	ranks 1, 2, 3 all end without rank 4's block
6	2 of 6	ranks 2, 3 end without ranks 4 and 5
7	3 of 7	rank 3 ends without ranks 4, 5, 6
10	6 of 10	ranks 2–7 end without ranks 8 and 9
12	4 of 12	ranks 4–7 end without ranks 8–11

At $p = 5$, rank 1's partners are $1 \oplus 1 = 0$, $1 \oplus 2 = 3$ and $1 \oplus 4 = 5$; the last does not exist, so rank 1 never learns anything from rank 4 and finishes holding four blocks out of five. Nothing in the

algorithm notices. There is no exception, no absentee list, and no shortfall in the message count — rank 1 simply returns a list with a `None` in it. The guard is not a remainder path; it is a correctness hole that would be silent.

The implementation closes it by substitution rather than by refusal:

```
if algorithm == "recursive_doubling" and (p & (p - 1)) != 0:  algorithm = "bruck"
```

The docstring calls this "the standard 'extra ranks' correction ... implemented here by falling back to Bruck", which is a fair description of the effect and slightly generous about the mechanism: MPI's extra-ranks correction adapts the algorithm, whereas this replaces it. The substitution is safe — Bruck has the same closed form and is strictly cheaper — and it is silent, which is the part with a consequence for the archive.

5.2 Why the sweep has nine runs and not twenty-one

Because the substitution happens before `_Tracker` is constructed, the emitted `coll.allgather` record carries `algorithm: "bruck"`. A sweep run directed at `recursive_doubling` at $p = 10$ would therefore have produced a run *named* `coll-allgather-recursive_doubling-10` whose every event said `bruck`: the two families would have shared twelve identical members, and any analysis that grouped by run name would have double-counted Bruck and attributed its behaviour to an algorithm that did not execute.

`experiments/microbench.py` avoids this with a single explicit condition in `bench_collectives`:

```
if alg == "recursive_doubling" and (p & (p - 1)) != 0 and op == "allgather":  continue
```

It is the only algorithm-specific skip in the whole sweep — `allreduce` and `scan` under the same algorithm name run at all 21 sizes, because their implementations have genuine remainder handling rather than a substitution. Checking the manifest confirms the skip took effect: the only `coll-allgather-recursive_doubling-*` runs in existence are at $p \in \{2, 4, 8, 16, 32\}$, nine of them across two campaigns, and every one records `algorithm: recursive_doubling`.

So the nine-versus-twenty-one asymmetry is not missing data and not a harness limitation. It is the applicability boundary of the algorithm, recorded by declining to fabricate measurements outside it. A reader comparing family sizes across the twenty-two packages should read a short family as a statement about the algorithm.

5.3 One place the restriction is not enforced

`cost.FORMULAS` has no notion of applicability, so `predict("allgather", "recursive_doubling", 5, n)` returns a perfectly well-formed prediction: 15 messages in 3 rounds. It happens to be numerically harmless, because that is also Bruck's prediction, so the model is never wrong about what a caller who asked for recursive doubling will actually pay — it is describing the substituted algorithm by accident.

`cost.best_algorithm` has the same gap and is protected by the same accident plus one more. It scores every candidate and sorts `(score, alg)` pairs; since Bruck and recursive doubling score identically on time, price and volume at every p, the tie is broken alphabetically and `bruck` always wins. The selector therefore never recommends an algorithm that cannot run — because `b` precedes

`r`. That is not a guard, and if a future edit made recursive doubling's formula marginally cheaper (a correct edit, incidentally, since it needs no final rotation) the selector would begin recommending it at sizes where it is illegal. Worth a guard.

6 When to choose this algorithm

6.1 Never, on the evidence here

At powers of two, recursive doubling and Bruck send the same number of messages in the same number of rounds carrying the same number of blocks. The logged figures are identical at all five sizes:

	messages		rounds		wire tokens		largest message (tok)	
p	rec. dbl.	bruck	rec. dbl.	bruck	rec. dbl.	bruck	rec. dbl.	bruck
2	2	2	1	1	14	8	7	4
4	8	8	2	2	84	44	14	7
8	24	24	3	3	384	200	27	14
16	64	64	4	4	1616	832	53	27
32	160	160	5	5	6624	3360	106	53

The wire columns are the entire difference, and they come from one line each. Recursive doubling sends `{str(i): v for i, v in have.items()}` — a dictionary that names each block's owner — so every block carries its key in every round it is forwarded. Bruck sends `local[:n_send]`, a bare list, and recovers ownership from position with a final rotation. The result is a factor of 1.97 on total wire volume at $p = 32$ and a factor of exactly two on the per-round message size: recursive doubling's sequence $7, 14, 27, 53, 106$ is Bruck's $4, 7, 14, 27, 53$ shifted by one round.

Two consequences. First, the price axis: at this benchmark's one-token blocks the keys dominate, and at realistic block sizes they would not, so the factor of two is specific to $n = 1$ — but the sign is not, because the overhead is per-block rather than per-message and does not amortise away. Second, the context axis: recursive doubling's final round at $p = 32$ delivers 16 blocks in one message and pays the key overhead on all of them, so if per-message size is a rank's binding constraint, recursive doubling hits it sooner than Bruck by a factor of two.

Against that, recursive doubling saves the final rotation in `algorithms.allgather`, which is $O(p)$ pointer moves and costs nothing measurable in runs whose whole duration is 0.247 s for 320 fabric operations. So the ledger reads: identical on messages and rounds, twice as expensive on the wire, applicable at 9 of 21 sizes, saves an operation that is free. There is no (p, n) region in which it should be chosen, which is why `allgather` defaults to `bruck` above $p = 3$ and substitutes it silently below.

6.2 What the family is worth, then

Two things, neither of them a recommendation.

It is a boundary measurement. Nine runs establish that the algorithm is exactly correct and exactly on its closed form where it is legal, which is the necessary complement to the simulation in §2.1 showing it is silently wrong where it is not. Together those say the guard is in the right place: not conservative, not permissive.

It is also a control for the sibling comparison. Because recursive doubling and Bruck are the same communication pattern up to encoding, the difference between their logged wire volumes isolates the cost of carrying rank indices in an allgather — a quantity no closed form in `cost.py` reports, since all three allgather entries give volume as $p(p-1)n$ and none models framing. At $p = 32$ that cost is 3,264 tokens, or 3.3 tokens per block per hop. For a harness moving prose artefacts it is negligible; for one moving many small structured records it is the whole cost, and this family is where the number comes from.

7 Threats to this reading

No agents in any of these runs. All nine record `executors: {none: p}`, zero agent calls, zero tokens in and out, $0.0000, `total_busy_s: 0` and `idle_fraction: 1.0`. `bench_collectives` calls `comm.allgather` with `algorithm="recursive_doubling"` and returns, so the viewer's timeline shows message ticks and lifecycle diamonds and no work bars. Every statement here is about communication structure, and nothing about model latency or agent behaviour follows from it.

The incompleteness table in §2.1 is a simulation, not a measurement, and it has to be. No run at a non-power-of-two p exists for this algorithm, so I reproduced the block-set propagation from the implementation's own partner rule and guard in a few lines of Python and counted which ranks end short. That is sound as an argument about the code, and it is corroborated by the fact that the implementation refuses to take the path. But it is not evidence from the archive, and it would be strengthened — or contradicted — by a run with the substitution disabled, which the archive does not contain and which I have not created.

Nine points, five distinct sizes, and no ability to see a step. Because the measured grid is $\{2, 4, 8, 16, 32\}$ every point lies at a bracket boundary, so this family cannot test the proposition that the round count is flat within a bracket, cannot show the 33% message-count jump just past a power of two, and cannot distinguish $\lceil \log_2 p \rceil$ from $\log_2 p$ or from several other functions that agree on powers of two. Those propositions are tested in the Bruck family, over the same range, with the same closed form. Any claim in this document about the shape of the curve is imported from there.

The wire-volume comparison is the one substantive number and it is measured at $n = 1$. 6,624 against 3,360 tokens at $p = 32$ is exactly what crossed the fabric, but with a two-character block the dictionary key is a comparable size to the block. The derived figure I would want — tokens of overhead per block per hop — comes out at 3.3, and that is a property of JSON serialisation of short keys rather than of the algorithm. A run with realistic artefacts would give a much smaller relative figure and roughly the same absolute one.

The `best_algorithm` tie-break argument is about a hazard, not an observed failure. I checked that the selector returns `bruck` at $p = 4, 8, 16, 32$ on all three objectives, and that it does so because the scores tie and `"bruck" < "recursive_doubling"` lexicographically. That the protection is incidental is a reading of `scored.sort()`, and no run in the archive was harmed by it. It is a note for whoever edits the formulas next, not a finding about the sweep.